

Módulo 05 - Reservas e Ciclo Operacional

Objetivo

Estudar a parte mais crítica do domínio: disponibilidade por período, reserva de unidades físicas, confirmação administrativa, início de locação, retirada e liberação manual.

Commits Estudados

- `5ca95fe` - reserva de unidade com validação de disponibilidade por período.
- `179ff89` - período de locação no pedido.
- `aa5286e` - reserva unidades físicas do pedido.
- `2ed40c5` - confirmação administrativa do pedido.
- `9040731` - início e retirada da locação.
- `c769c2a` - liberação manual de unidades.
- `33f8c10` - corrige ações disponíveis de pedido.

Aula 1 - Disponibilidade Por Período

O commit `5ca95fe` adiciona consulta de disponibilidade por período.

Código para ler:

- `backend/catalogo/services.py`
- `backend/catalogo/serializers.py`
- `backend/catalogo/views.py`
- `backend/pedidos/models.py`
- `backend/catalogo/tests.py`

Endpoint de brinquedo:

- `GET /api/brinquedos/<id>/disponibilidade/?data_inicio=...&data_fim=...`

Também aparecem endpoints de disponibilidade para:

- kits prontos;
- kits personalizáveis.

A regra central:

```
.exclude(
    reservas__status=ReservaUnidade.Status.ATIVA,
    reservas__data_inicio__lt=data_fim,
    reservas__data_fim__gt=data_inicio,
)
```

Essa condição representa conflito de períodos.

Lição:

- Períodos sobrepostos conflitam.
- Períodos encostados não conflitam.
- Unidade com status diferente de `DISPONIVEL` não entra na contagem.

Testes para estudar:

- `test_periodos_sobrepostos_conflitam`
- `test_periodos_encostados_nao_conflitam`
- `test_unidades_com_status_nao_disponivel_nao_contam`
- `test_consulta_nao_cria_reservas_nem_altera_status_da_unidade`

Aula 2 - Período de Locação no Pedido

O commit `179ff89` adiciona `data_inicio_locacao` e `data_fim_locacao` ao pedido.

Código para ler:

- `backend/pedidos/models.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/services.py`
- `backend/pedidos/tests.py`

Regra:

- data final deve ser posterior à inicial;
- pedido convertido salva o período;
- erro de período inválido não cria pedido;
- erro de período inválido não converte carrinho.

Testes para estudar:

- `test_conversao_valida_salva_data_inicio_locacao`
- `test_conversao_valida_salva_data_fim_locacao`
- `test_conversao_rejeita_periodo_locacao_invalido`
- `test_erro_de_periodo_invalido_nao_converte_carrinho`

Aula 3 - Reserva de Unidades Físicas

O commit `aa5286e` implementa a reserva administrativa das unidades.

Código para ler:

- `backend/pedidos/services.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/views.py`
- `backend/pedidos/urls.py`
- `backend/pedidos/tests.py`

Endpoint:

- `POST /api/admin/pedidos/<id>/reservar-unidades/`

Pontos reais:

- Apenas admin acessa.
- Pedido precisa estar em status reservável.
- Pedido precisa ter período válido.
- Service monta demandas por brinquedo a partir dos itens do pedido.
- Reserva usa unidade física livre.
- Falta de estoque causa rollback total.
- Segunda chamada compatível é tratada com idempotência.

Trecho conceitual:

```
@transaction.atomic
def reservar_unidades(pedido):
    pedido = Pedido.objects.select_for_update().get(id=pedido.id)
```

No código atual, o padrão aparece por `@transaction.atomic` e `select_for_update()`.

Testes para estudar:

- `test_admin_reserva_unidades_de_pedido_com_brinquedo_avulso`
- `test_reserva_kit_festa_usa_snapshot_se_catalogo_for_alterado`
- `test_falta_de_estoque_gera_rollback_total_sem_reserva_parcial`
- `test_bloqueia_overbooking_por_reserva_conflitante_revalidada`

Aula 4 - Confirmação Administrativa

O commit `2ed40c5` cria a confirmação administrativa.

Código para ler:

- `backend/pedidos/models.py`
- `backend/pedidos/services.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/views.py`
- `backend/pedidos/tests.py`

Endpoint:

- `POST /api/admin/pedidos/<id>/confirmar/`

Regras reais:

- pedido precisa estar `RESERVADO`;
- pedido precisa ter aceite de contrato;
- pedido precisa ter período válido;
- pedido precisa ter reservas ativas;
- reservas precisam ser compatíveis com demanda e período;
- confirmação salva `confirmado_em` e `confirmado_por`.

Teste-chave:

- `test_pedido_sem_aceite_de_contrato_nao_pode_ser_confirmado`

Esse teste conecta diretamente a regra do projeto: pedido não confirma sem contrato aceito.

Aula 5 - Início e Retirada da Locação

O commit `9040731` adiciona duas ações operacionais:

- iniciar locação;
- registrar retirada.

Código para ler:

- `backend/pedidos/models.py`
- `backend/pedidos/services.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/views.py`
- `backend/pedidos/tests.py`

Endpoints:

- `POST /api/admin/pedidos/<id>/iniciar-locacao/`
- `POST /api/admin/pedidos/<id>/registrar-retirada/`

Fluxo real:

1. Pedido confirmado inicia locação.
2. Unidades reservadas mudam para `EM_LOCACAO`.
3. Retirada encerra reservas ativas.
4. Unidades vão para `HIGIENIZACAO`.
5. Pedido muda para `RETIRADO`.

Regra crítica:

- Item devolvido não volta automaticamente para disponível.

Teste para estudar:

- `test_admin_registra_retirada_de_pedido_em_locacao`
- `test_registrar_retirada_faz_rollback_se_unidade_estiver_inconsistente`

Aula 6 - Liberação Manual

O commit `c769c2a` adiciona liberação manual de unidades.

Código para ler:

- `backend/catalogo/services.py`
- `backend/catalogo/views.py`
- `backend/catalogo/urls.py`

- `backend/catalogo/tests.py`

Endpoint:

- `POST /api/admin/unidades/<id>/liberar-disponibilidade/`

Regra real:

- Admin só pode liberar unidade em `HIGIENIZACAO` ou `STANDBY`.
- Unidade em locação, reservada, manutenção, baixada ou já disponível retorna erro.
- Liberação não altera reservas nem pedidos.

No service atual:

```
STATUS_LIBERAVEIS = {
    UnidadeBrinquedo.Status.HIGIENIZACAO,
    UnidadeBrinquedo.Status.STANDBY,
}
```

Testes para estudar:

- `test_admin_libera_unidade_em_higienizacao_para_disponivel`
- `test_unidade_em_locacao_nao_pode_ser_liberada`
- `test_liberacao_nao_altera_reservas_existentes`

Aula 7 - Correção das Ações Disponíveis

O commit `33f8c10` corrige ações disponíveis no pedido.

Código para ler:

- `backend/pedidos/services.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/tests.py`

Contexto real do código:

- A UI admin depende de `acoes_disponiveis`.
- A lista de ações não pode sugerir uma transição inválida.
- O backend recalcula essas ações conforme status, contrato, reservas e período.

Testes para estudar:

- `test_acoes_disponiveis_incluem_reservar_para_pedido_reservavel`
- `test_acoes_disponiveis_nao_incluem_reservar_sem_itens_ou_periodo`
- `test_acoes_disponiveis_dos_principais_status`

Revisão do Módulo

Você deve conseguir responder:

- Qual diferença entre consultar disponibilidade e reservar unidade?
- Por que reserva precisa de transação?
- Onde o sistema impede overbooking?
- Por que retirada manda unidade para higienização?
- Por que liberação manual não deve mexer em pedido ou reserva?

Exercício Prático

Abra `git show aa5286e` e encontre:

- o service de reserva;
- o endpoint admin;
- os testes de rollback e overbooking.

Depois abra `git show 9040731` e desenhe a máquina de estados do pedido e da unidade.